Diya Nagdev
D15A-04
ML-DL exp-01

Experiment Title

Implementation and Analysis of Linear and Logistic Regression on a Real-World Dataset Using Google Colab

1. Dataset Source

The dataset used for this experiment is the **Fake Social Media Account Detection**

Dataset, obtained from Kaggle.

Source Link:
https://www.kaggle.com/datasets/free4ever1/instagram-fake-spammer-genuine-accounts?resource=download
This dataset contains Instagram profile features used to classify accounts as fake or genuine.

2. Dataset Description

The **Fake Social Media Account Detection Dataset** consists of various attributes related to social media profiles. The dataset is designed for **binary classification**, where the objective is to identify whether an account is fake.

Dataset Characteristics

- **Features may include:**
  - number of followers
  - number of following
  - number of posts
  - account activity indicators
  - profile characteristics
- **Target Variable**
  - `fake`

    - **0 → Genuine Account**
    - **1 → Fake Account**
- **Data Type:** Numerical and categorical features
- **Nature of Dataset:** Real-world structured data related to social media profiles.

This dataset helps train machine learning models to detect suspicious or fake accounts on social platforms.

3. Mathematical Formulation of the Algorithm

3.1 Linear Regression

Linear Regression models the relationship between independent variables and a continuous dependent variable using a linear equation.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Where:

- **y** = predicted output
- **$x_1, x_2 \dots x_n$** = input features (e.g., followers, posts, engagement)
- **β_0** = intercept
- **$\beta_1 \dots \beta_n$** = model coefficients

The objective of Linear Regression is to minimize the **Mean Squared Error (MSE)** between predicted and actual values.

3.2 Logistic Regression

Logistic Regression is used for **binary classification problems**, such as detecting whether a social media account is fake or real.

The model uses the **sigmoid function** to convert predictions into probabilities.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

- **$\sigma(z)$** = predicted probability
- **z** = linear combination of input features

The predicted probability is converted into a class label:

- **$P(y = 1) > 0.5 \rightarrow$ Fake Account**
- **$P(y = 0) \leq 0.5 \rightarrow$ Genuine Account**

4. Algorithm Limitations

Linear Regression Limitations

- Assumes a linear relationship between variables
- Sensitive to outliers
- Not suitable for classification problems
- Limited ability to model complex patterns

Logistic Regression Limitations

- Works best when classes are linearly separable
- Performance may decrease with highly correlated features
- Sensitive to outliers
- Requires proper feature scaling for better performance

5. Methodology / Workflow

```
[2]
✓ 2s
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import LinearRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.metrics import mean_squared_error
```

```
import pandas as pd

df = pd.read_csv("train.csv")
df.head()
```

	profile pic	nums/length username	fullname words	nums/length fullname	name==username	description length	external URL	private	#posts	#followers	#follows	fake
0	1	0.27	0	0.0	0	53	0	0	32	1000	955	0
1	1	0.00	2	0.0	0	44	0	0	286	2740	533	0
2	1	0.10	2	0.0	0	0	0	1	13	159	98	0
3	1	0.00	1	0.0	0	82	0	0	679	414	651	0
4	1	0.00	2	0.0	0	0	0	1	6	151	126	0

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 576 entries, 0 to 575
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   profile pic           576 non-null    int64
 1   nums/length username  576 non-null    float64
 2   fullname words        576 non-null    int64
 3   nums/length fullname  576 non-null    float64
 4   name==username        576 non-null    int64
 5   description length     576 non-null    int64
 6   external URL          576 non-null    int64
 7   private               576 non-null    int64
 8   #posts                576 non-null    int64
 9   #followers            576 non-null    int64
 10  #follows              576 non-null    int64
 11  fake                  576 non-null    int64
dtypes: float64(2), int64(10)
memory usage: 54.1 KB
```

```python
X = df.drop("fake", axis=1)
y = df["fake"]
```

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```python
from sklearn.linear_model import LogisticRegression

log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

```
LogisticRegression
LogisticRegression(max_iter=1000)
```

```python
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9224137931034483
```

```python
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.90      0.97      0.93        63
           1       0.96      0.87      0.91        53

    accuracy                           0.92       116
   macro avg       0.93      0.92      0.92       116
weighted avg       0.93      0.92      0.92       116
```

```python
from sklearn.linear_model import LinearRegression

lin_model = LinearRegression()
lin_model.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

6. Performance Analysis

Logistic Regression Performance

Evaluation metrics used:

- Accuracy
- Precision
- Recall
- F1-Score

Logistic Regression effectively classifies social media accounts as **fake or genuine** based on profile features.

```
y_pred = log_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.5106677750069272

Classification Report:
               precision    recall  f1-score   support

        fake       0.52      0.53      0.53      1846
        real       0.50      0.49      0.49      1763

    accuracy                           0.51      3609
   macro avg       0.51      0.51      0.51      3609
weighted avg       0.51      0.51      0.51      3609
```

Linear Regression Performance

Evaluation metric used:

- **Mean Squared Error (MSE)**

Linear Regression can be used to predict numerical attributes such as **engagement metrics or follower counts**.

```
y_pred_l = lin_model.predict(X_test_l)

mse = mean_squared_error(y_test_l, y_pred_l)
print("Mean Squared Error:", mse)
```

```
Mean Squared Error: 7181.175796554777
```

Final Conclusion

The experiment successfully implemented **Linear Regression and Logistic Regression** on a **social media account dataset** using Google Colab. Logistic Regression proved effective in identifying fake accounts, while Linear Regression demonstrated the ability to predict numerical social media metrics. Proper preprocessing and feature selection improved the overall model performance.